

Creating a Service-Oriented Architecture for Scalable Use of Language Models in Commercial Solutions

ModelWeave: deployed document-grounded agent platform with database, workflows, approvals, tasks, and DOCX reports

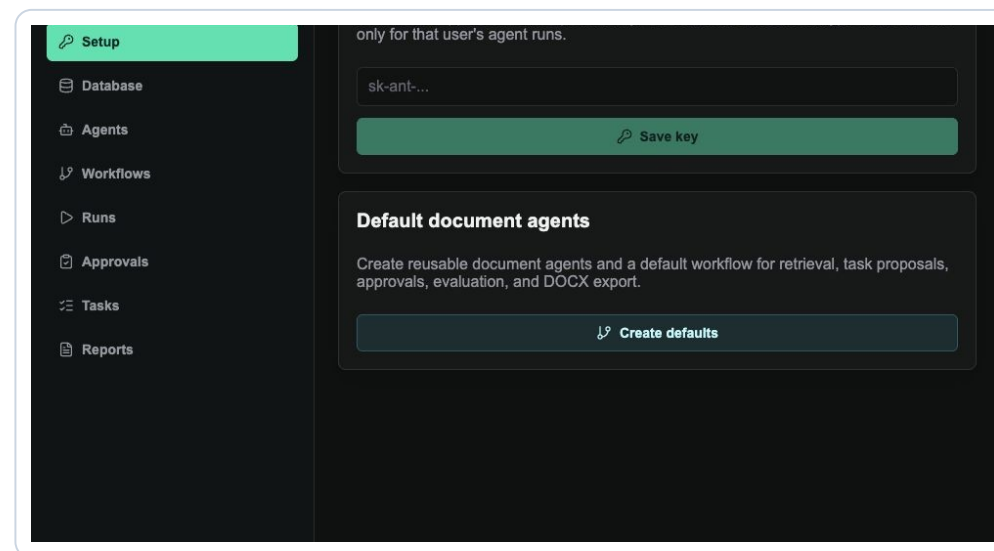
Student

Serhii Sotskyi, group AB221

Supervisor

Vira Oleksandrivna Hrishyna

National University "Odesa Polytechnic" | Odesa, 2026



Real deployed web app

Synthetic data only

DOCX outputs

Defense duration target: under 15 minutes

Production demo is linked at the end

Build a scalable LLM architecture, not a single chatbot

Goal

Design and evaluate a service-oriented architecture for scalable use of language models in commercial workflows.

Case study: ModelWeave, a document-grounded agent platform with RAG, reusable agents, workflows, approvals, audit events, tasks, and DOCX reports.

1

Theory

Review LLMs, RAG, vector search, SOA, and multi-agent systems.

2

Requirements

Define functional and non-functional needs for a document-agent platform.

3

Mathematics

Formalize retrieval, embeddings, workflow graphs, approvals, and evaluation.

4

Implementation

Build the full-stack application using Next.js, FastAPI, Supabase, Qdrant, and Anthropic.

5

Validation

Run synthetic document workflows and verify reports, events, citations, approvals, and tasks.

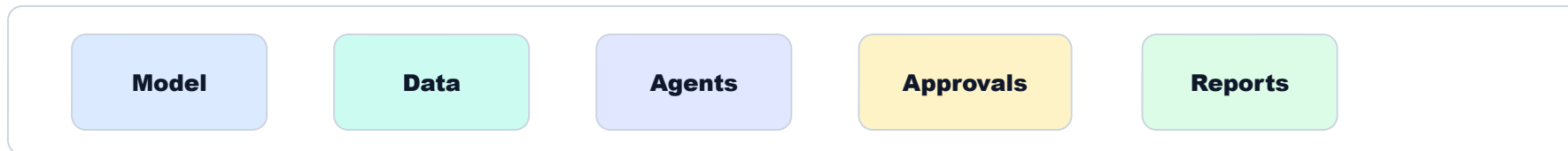
Commercial LLM adoption fails when model output is not connected to workflow

A chatbot can answer

- Respond to a prompt without persistent business state
- Summarize text without proving every operational decision
- Draft recommendations but not safely execute team workflows
- Give fluent output while hiding retrieval, risk, and audit context

A commercial system must control

- Users, documents, model keys, permissions, and storage
- Retrieval, citations, agents, approvals, and audit events
- Typed workflow nodes that turn outputs into proposed actions
- Exported deliverables that can be reviewed outside the app



ModelWeave connects these parts into one controlled workflow.

The prototype combines retrieval, agent workflows, and governance controls

SOA

RAG

Embeddings

Vector search

Multi-agent graph

Approval gates

BYOK

Approach	Workflow control	Grounding	Mutation safety	Fit for thesis
Generic chatbot	Conversation only	Weak or implicit	None	Low
Simple RAG report tool	Report generation	Document citations	Limited	Medium
Generic agent builder	Flexible automation	Depends on setup	Often optional	Medium
ModelWeave	Typed document workflow	RAG citations	Mandatory approvals	High

Key idea: language model calls become controlled services inside a workflow, not isolated answers.

RAG turns documents into vectors, then ranks evidence before generation

Document corpus

$$D = \{d_1, d_2, \dots, d_n\}; \quad d_i = \{c_{i,1}, c_{i,2}, \dots, c_{i,m_i}\}; \quad \phi(c_j) = e_j \in \mathbb{R}^d$$

Documents are parsed, chunked, embedded, and stored as searchable vectors.

Similarity

$$\text{sim}(q, c_j) = (v_q \cdot e_j) / (||v_q|| ||e_j||)$$

Cosine similarity ranks text chunks by semantic closeness to the query.

Top-k retrieval

$$R_k(q) = \arg \text{top-}k \ c_j \in C \ \text{sim}(q, c_j)$$

Only the most relevant chunks are injected into the agent context.

RAG generation

$$P(y|q, R_k, a) = \prod_t P(y_t \mid y_{<t}, q, R_k, a)$$

The answer depends on the question, retrieved evidence, and agent role.

Retrieval metrics

$$\text{Precision@}k = |R_{\text{el}}(q) \cap R_k(q)| / k$$

Citation coverage is a practical MVP proxy for retrieval quality.

Embedding dimension
 d

Retrieval depth k

Chunk size b

Evidence citations

Agent role a

Agent workflows are typed graphs with state, approvals, cost, and quality scores

Workflow graph

$$G = (V, E, \tau, \sigma)$$

$\tau(v) \in \{\text{retrieve, agent, create_task, approval, evaluate, export_docx}\}$

$\sigma(v) \in \{\text{waiting, running, completed, approval_required, failed}\}$



Approval constraint

$\text{mutate}(x) \Rightarrow$
 $\text{approval}(x) = \text{approved}$

Quality score

$$S = (wcC + waA + wrR + wmM) / (wc + wa + wr + wm)$$

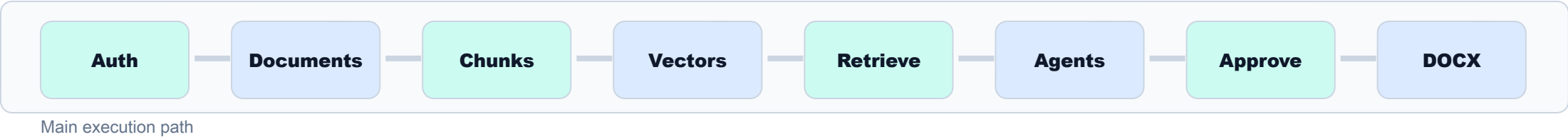
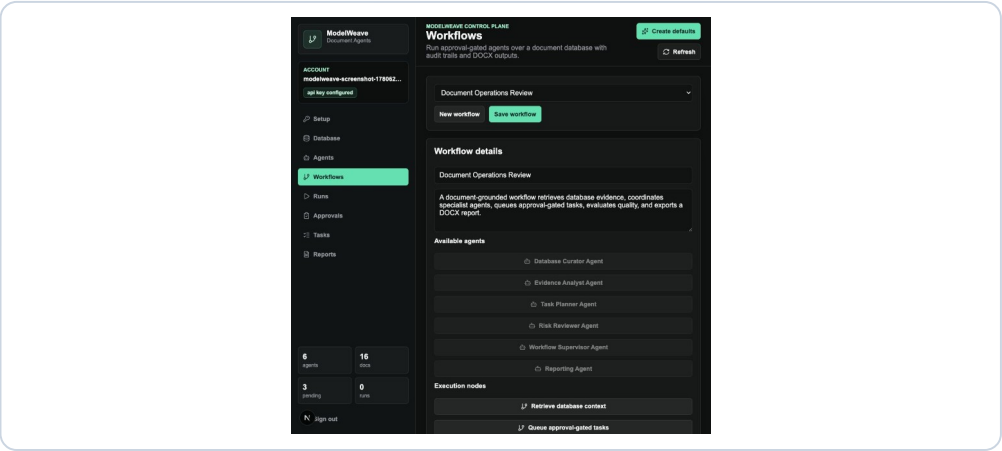
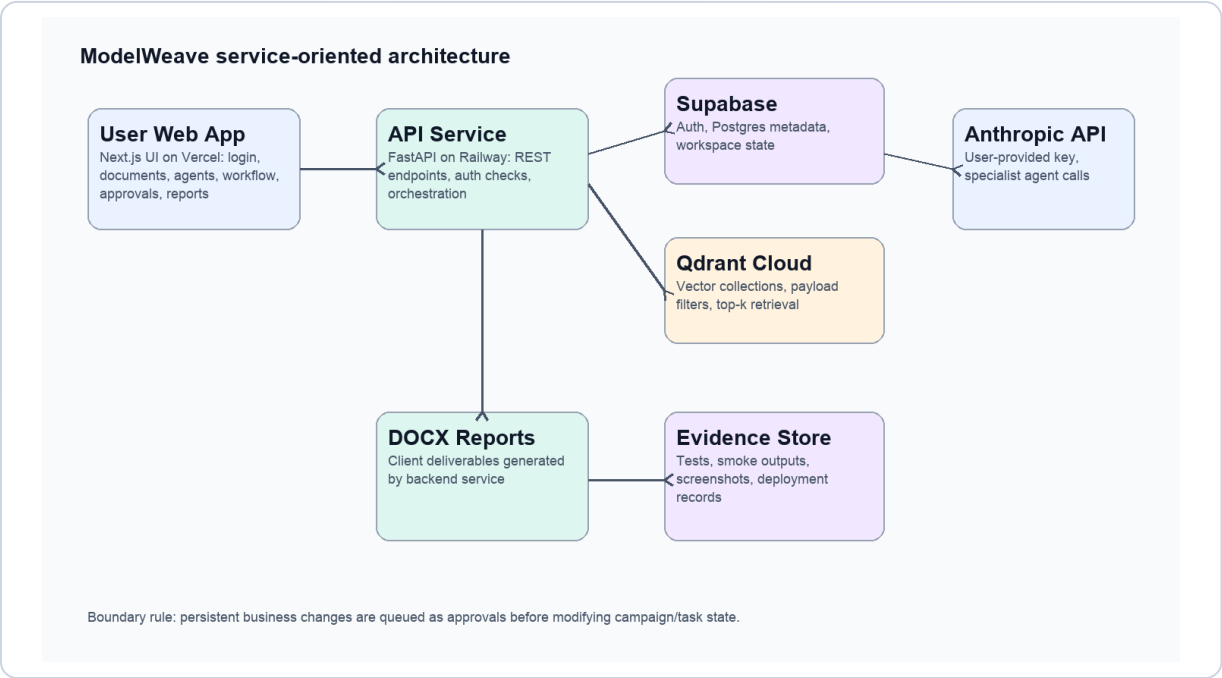
Token cost

$$\text{Cost} = \sum_j (\text{pinTin}, j + \text{poutTout}, j)$$

Applied-math value

The formulas connect product behavior to measurable parameters: corpus size $|C|$, vector dimension d , retrieval depth k , agent count $|V_{\text{agent}}|$, pending approvals, latency, and run quality.

Documents become retrievable context; agents turn context into approval-gated



RESULTS

The thesis result is a deployed, tested full-stack prototype

16

synthetic documents seeded

8

citations in clean agent run

21+

recorded run events in QA

3+

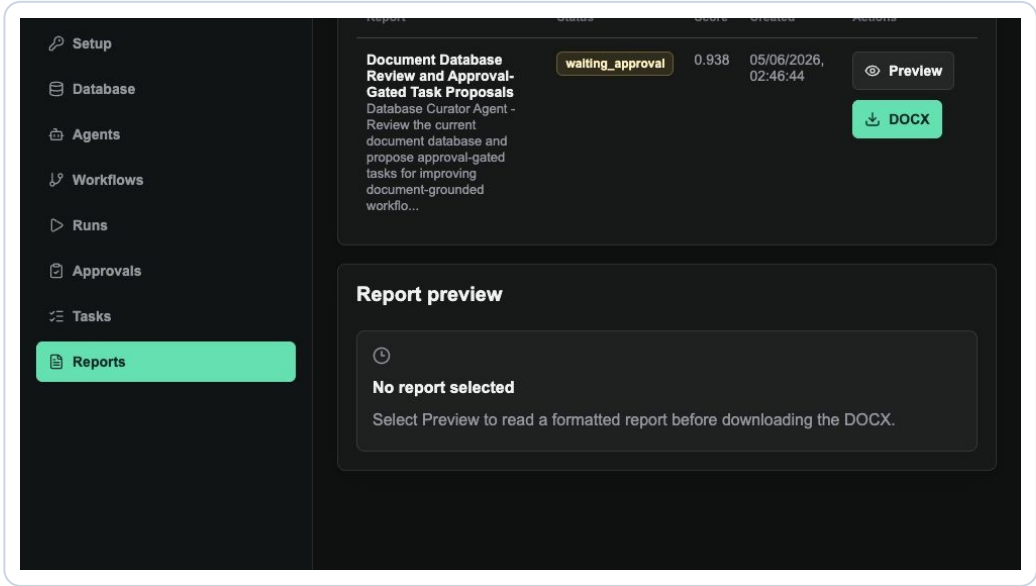
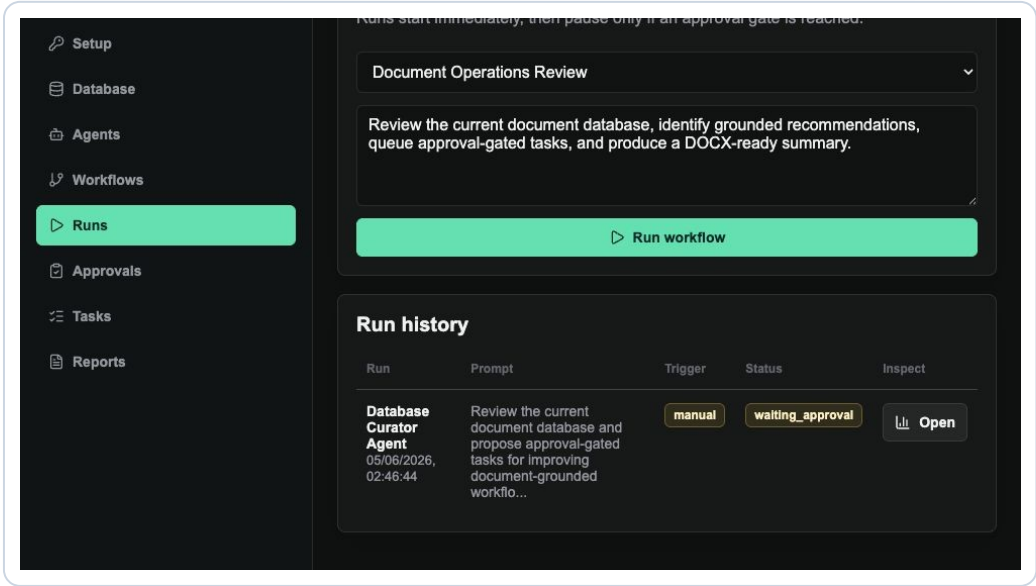
approval records created

0

raw Markdown markers in DOCX

1.0

run evaluation score



Production frontend: <https://modelweave-two.vercel.app>

Backend: <https://api-production-e70a9.up.railway.app>

Repository: [GitHub public](#)

ModelWeave proves that LLM capabilities can be organized as a managed commercial workflow

SOA is the backbone

Separate services make the system deployable, testable, and easier to scale.

RAG grounds the output

Documents are chunked, embedded, retrieved, and cited before agents produce recommendations.

Agents represent roles

Document reviewer, curator, task planner, QA, and report writer agents cooperate.

Approvals reduce risk

Persistent business changes are proposed first and applied only after human approval.

The prototype is practical

The result is a deployed web app with authentication, BYOK, uploads, workflows, audit events, and DOCX reports.

Future work: durable workflow queue, external connectors, richer evaluation, and organization-level permissions.

Open the deployed ModelWeave prototype and run a document-agent workflow



<https://modelweave-two.vercel.app>

Demo path

- 1 Sign in**
Register or confirm the user session through Supabase Auth.
- 2 Add model key**
Save the user's own Anthropic API key for agent execution.
- 3 Load corpus**
Seed the synthetic academic document set.
- 4 Run workflow**
Run the default Document Operations Review workflow.
- 5 Review output**
Inspect timeline, citations, approvals, tasks, evaluation, and DOCX report.

Backend: <https://api-production-e70a9.up.railway.app>

Repository: <https://github.com/serhiiSotskyi/multi-agent-knowledge-platform>